C++ 基础专题

switch 语句与三目运算符

入门讲义与 24 道超级基础配套练习

目标：掌握基础知识点和最基本应用

法、代码阅读与一步应用。

资料日期：2026 年 8 月 19 日

编译方式：XeLaTeX

目录

1 使用说明 **3**

2 图片内容重新编码 **3**

2.1 图片 1：switch 语句与三目运算符 3

2.2 图片 2：三目运算符补充特点 4

3 原图问题与必要纠错 **4**

3.1 纠错 1：switch 条件不只是“整型或者字符型” 4

3.2 纠错 2：“不可以是一个区间”的真正原因 5

3.3 纠错 3：“执行效率高”不是语言层面的绝对保证 5

3.4 纠错 4：没有 break 时，不只是“自动执行下一项” 5

3.5 纠错 5：default 和最后一个 break 都不是强制存在 5

3.6 纠错 6：“三目运算符返回变量”不能无条件成立 5

4 知识点清单 **6**

5 核心概念详解 **6**

5.1 switch：离散多分支选择 6

5.1.1 基本定义 6

5.1.2 通俗解释 6

5.1.3 最基础语法 7

5.1.4 每个符号和关键字的含义 7

5.1.5 适用范围 7

5.1.6 最基础使用步骤 7

5.1.7 极其简单示例 8

5.1.8 初学者常见错误 8

5.2 break 与 fall-through 9

5.2.1 break 的作用 9

5.2.2 没有 break 会发生什么 9

5.3 三目（条件）运算符?: 9

5.3.1 基本定义 9

5.3.2 通俗解释 9

5.3.3 三个部分分别是什么 10

5.3.4 最基础使用步骤 10
5.3.5 什么时候适合用 10
5.3.6 “可以继续赋值”的正确理解 10

6 switch 与三目运算符的区别 11

7 最基础示例汇总 11

7.1 示例 1：用 switch 选择菜单 11
7.2 示例 2：观察贯穿 11
7.3 示例 3：用三目运算符求较大值 12
7.4 示例 4：条件表达式作为赋值目标 12

8 知识点—题目对应表 13

9 单项选择题（10 道） 14

10 多项选择题（5 道） 15

11 填空题（5 道） 16

12 基础解答题（4 道） 17

13 答案速查表 19

14 单项选择题详细解析 20

15 多项选择题详细解析 24

16 填空题详细解析 26

17 基础解答题详细解析 28

18 高频易错点总结 31

19 基础解题步骤总结 31

19.1 看到 switch 题时 31
19.2 看到三目运算符时 31

20 参考资料与核验来源 32

21 最终质量检查记录 33

1 使用说明

本资料根据两张 C++ 手写笔记图片重新编码，主题集中在两个知识点：**switch 多分支选择语句**与**三目（条件）运算符 ?:**。资料的定位是“第一次学完之后的基础巩固”，因此不讨论复杂模板、重载运算符、复杂值类别推导等进阶内容。

建议按以下顺序使用：先看“图片内容重新编码”和“必要纠错”，再学习两个核心知识点；完成全部 24 道题后，再翻到答案与详细解析。题目部分不会用颜色、加粗或其他方式暗示正确答案。

学习完成后应达到的最低目标

- 能写出最基本的 `switch--case--default` 结构；
- 知道 `break` 在 `switch` 中通常用于阻止继续向下执行；
- 知道 `case` 对应的是**离散的常量值**，不能直接写“某个区间”；
- 能写出 `condition ? expr2 : expr3`；
- 能判断条件为真或假时，到底会求值第二个还是第三个表达式；
- 能理解手写笔记中“条件运算符可以继续赋值”这句话成立的具体条件。

2 图片内容重新编码

2.1 图片 1：switch 语句与三目运算符

以下先尽量忠实保留原笔记的表达方式，不在本小节直接悄悄改写。

二、switch 语句

1. 作用：执行多条任务分支语句。[1]

2. 语法：

```
switch (expression)
{
    case result1: statement; break;
    case result2: statement; break;
    ...
    default: statement; break;
}
```

原图在第一个 `break` 旁有红字说明：**“要不然自动执行 case 结果 2”**。

3. 特点：

① 缺点：判断时候只能是整型或者字符型，不可以是一个区间；

[1] “任务分支”处字迹略潦草，按原图可辨内容转录；从上下文看其核心含义是“多分支选择”。

② 优点：结构清晰，执行效率高。

4. 注意：

① `switch` 语句中表达式类型只能是整型或者字符型；

② `case` 里如果没有 `break`，程序会一直向下执行。

三、三目运算符

1. 作用：通过三目运算符实现简单的判断。

2. 语法：

表达式 1 ? 表达式 2 : 表达式 3

3. 解释：

- 如果表达式 1 的值为真，执行表达式 2，并返回表达式 2 的结果；
- 如果表达式 1 的值为假，执行表达式 3，并返回表达式 3 的结果。

2.2 图片 2：三目运算符补充特点

原图顶部写有“C++”“§1. 选择语句”，随后继续“三、三目运算符”。新增内容为：

4. 特点：在 C++ 中三目运算符返回的是变量，可以继续赋值。

3 原图问题与必要纠错

说明

手写笔记的主线思路是正确的，但有几处为了便于入门而做了简化，其中两处如果按字面理解会不够严谨。下面先指出原写法，再给出适合初学者的规范表述。

3.1 纠错 1：switch 条件不只是“整型或者字符型”

原图写法：“`switch` 语句中表达式类型只能是整型或者字符型。”

问题：这个说法过窄。按 C++ 语言规则，`switch` 条件可以是**整数类型**、**枚举类型**，也可以是能够按规则转换到整数或枚举类型的类类型。`char` 本身属于整数类型的一种，因此“字符型可以”没有错，但并不是只有 `int` 和 `char` 两类。[1][2]

适合入门阶段的记法

先记成：**`switch` 适合对整数、字符、枚举等“离散值”做多分支选择；不能直接拿浮点数或字符串作为普通 `switch` 条件来逐个 `case` 匹配。**

3.2 纠错 2：“不可以是一个区间”的真正原因

原图写法：“不可以是一个区间。”

这个结论在基础语境下是对的，但更规范的原因是：case 后面需要的是能在编译期确定的**常量表达式值**，例如 case 1:、case 'A':，而不能直接写 case x > 0: 表示一个运行时区间。[1]

例如，要判断分数是否在 $[90, 100]$，初学阶段更适合写：

```
if (score >= 90 && score <= 100) {
    // ...
}
```

而不是把“区间条件”硬塞进普通 case 标签。

3.3 纠错 3：“执行效率高”不是语言层面的绝对保证

原图写法：“优点：结构清晰，执行效率高。”

“结构清晰”通常成立；但“执行效率一定更高”不是 C++ 语言标准保证。编译器可能把某些 switch 优化成跳转表，也可能采用其他分支结构；实际性能取决于编译器、优化等级、case 分布和目标平台。因此，入门阶段应优先依据**代码表达是否清晰、是否适合离散多分支**来选择 switch，不要把“必然更快”当成定理。

3.4 纠错 4：没有 break 时，不只是“自动执行下一项”

原图红字：“要不然自动执行 case 结果 2。”

更准确地说：当某个 case 匹配后，如果后续没有遇到能够离开 switch 的控制语句（例如 break、return），控制流会继续向下穿过后续标签执行，这叫**贯穿（fall-through）**。它可能不只执行“下一项”，而是一直向后执行到 break、return 或 switch 末尾。[1][2][4]

3.5 纠错 5：default 和最后一个 break 都不是强制存在

default 是可选的。如果没有任何 case 匹配且也没有 default，那么该 switch 体内对应语句不执行。[1]

另外，如果 default 或某个 case 已经位于整个 switch 的最后，末尾的 break 常常可以省略，因为代码自然就走出了 switch。不过初学阶段为了保持结构一致，可以先保留。

3.6 纠错 6：“三目运算符返回变量”不能无条件成立

原图写法：“在 C++ 中三目运算符返回的是变量，可以继续赋值。”

更准确地说，条件运算符 ?: 的结果具有一定的**类型和值类别**。当第二、第三个操作数满足特定条件，例如它们是同类型的可修改左值时，整个条件表达式也可以表现为左值，因此可以放在赋值号左边。标准对这一点有严格的类型和值类别规则。[1][3]

最适合初学者记住的两个例子是：

```cpp
int a = 1, b = 2;
bool flag = true;

(flag ? a : b) = 100;   // valid: assigns to a

// (flag ? 10 : 20) = 100; // invalid: 10 and 20 are not variables
```

因此不要死记“?: 永远返回变量”，而应记成：**普通情况下它返回被选中分支的结果；在某些左右分支都是合适左值的情况下，整个条件表达式还能继续作为赋值目标**。

4 知识点清单

知识点	最低掌握要求
switch 的用途	根据一个离散条件值，在多个分支中选择执行路径
case	写出一个可匹配的常量值，例如 `case 1:`、`case 'A':`
default	没有任何 case 匹配时的可选兜底分支
break	结束当前最内层的 switch 或循环，防止继续向下贯穿
fall-through	匹配后没有离开 switch 时，控制流继续进入后面的标签语句
?: 的结构	`condition ? expr2 : expr3`
?: 的求值	条件真只求值第二项，条件假只求值第三项
条件表达式作为左值	仅在特定类型和值类别条件下成立，不是“永远返回变量”

5 核心概念详解

5.1 switch：离散多分支选择

5.1.1 基本定义

switch 是一种**选择语句（selection statement）**。它先求出条件的值，再把这个值与各个 case 的常量值比较；匹配到某个 case 后，从那个位置开始执行。若没有任何 case 匹配且存在 default，则进入 default。[1][2]

5.1.2 通俗解释

可以把 switch 想成“按编号去柜台”：

- 条件值是“你拿到的号码”；
- 每个 case 是一个固定编号的柜台；
- 编号相同就从那个柜台开始办事；
- 都对不上时，如果有 default，就去“其他情况”柜台。

5.1.3 最基础语法

```
switch (choice)
{
    case 1:
        statement1;
        break;
    case 2:
        statement2;
        break;
    default:
        defaultStatement;
        break;
}
```

5.1.4 每个符号和关键字的含义

写法	含义
`switch`	开始一个多分支选择结构
`(choice)`	条件；程序会求出它的值
`case 1:`	当条件值等于 1 时，从此处开始执行
`break;`	结束当前最内层 `switch`，跳到整个 `switch` 后面的语句
`default:`	所有 `case` 都不匹配时进入；它不是强制必须写
`{ }`	包住 `switch` 的主体语句

5.1.5 适用范围

入门阶段最适合以下场景：

- 菜单编号：1、2、3、4；
- 字符命令：'A'、'B'、'C'；
- 星期编号、状态编号；
- 枚举值等离散类别。

如果条件本质是“分数是否大于 90”“温度是否位于某区间”，通常 `if--else if--else` 更自然。

5.1.6 最基础使用步骤

1. 找出你要判断的一**个离散值**，放进 `switch(...)`；
2. 为每个具体值写一个 `case 常量:`；

3. 在每个分支写要执行的语句；
4. 如果不希望继续执行后面的分支，写 `break;`；
5. 需要处理“其他所有值”时再写 `default:`。

5.1.7 极其简单示例

```cpp
#include <iostream>
using namespace std;

int main() {
    int choice = 2;

    switch (choice) {
        case 1:
            cout << "A";
            break;
        case 2:
            cout << "B";
            break;
        default:
            cout << "Other";
            break;
    }
}
```

因为 `choice` 等于 2，所以匹配 `case 2:`，输出 `B`，随后 `break` 结束 `switch`。

5.1.8 初学者常见错误

1. 把 `case` 写成区间条件，例如 `case score >= 90:`；
2. 忘记 `break`，导致出现意外贯穿；
3. 以为 `default` 必须写；
4. 以为 `switch` 只能接 `int`，忘记 `char`、枚举等也属于常见可用类型；
5. 把 `case` 后面的冒号 `:` 写成分号 `;`。

一句记忆

switch 看“值”，case 列“点”，break 负责“停”，default 管“其他”。

5.2 break 与 fall-through

5.2.1 break 的作用

在 switch 内，break; 会终止当前最内层 switch，控制流转到整个 switch 后面的下一条语句。[1][4]

5.2.2 没有 break 会发生什么

看下面的代码：

```
int x = 1;
switch (x) {
    case 1:
        cout << "A";
    case 2:
        cout << "B";
        break;
    default:
        cout << "C";
}
```

x=1 时先进入 case 1 输出 A。由于这里没有 break，执行不会重新判断 case 2 是否匹配，而是**直接继续向下**，因此又输出 B，最后遇到 break 停止。输出结果是 AB。

高频误区

贯穿不是“再比较一次下一条 case”。一旦已经跳到匹配位置，后面的 case/default 标签本身不会自动阻挡控制流。

5.3 三目（条件）运算符?:

5.3.1 基本定义

条件运算符的基本形式是：

$$\text{条件} \ ? \ \text{表达式 2} \ : \ \text{表达式 3}$$

它一共有三个操作数，所以常被称为“三目运算符”或“条件运算符”。条件首先按布尔意义判断；条件为真，整个表达式取第二个表达式的结果；条件为假，取第三个表达式的结果。第二、第三个表达式中**只会求值一个**。[1][3]

5.3.2 通俗解释

它就是“二选一”的紧凑表达式：

如果条件成立，就选左边；否则选右边。

例如：

```
int a = 3;
int b = 5;
int maxValue = (a > b) ? a : b;
```

因为 a > b 为假，所以选择 b，最终 maxValue = 5。

5.3.3 三个部分分别是什么

部分	含义
表达式 1（条件）	决定走哪一边；按布尔意义判断真或假
表达式 2	条件为真时才求值，并作为结果来源
表达式 3	条件为假时才求值，并作为结果来源

5.3.4 最基础使用步骤

1. 先写一个能判断真假的条件；
2. 问自己：“条件为真时我要得到什么?”写在 ? 后面；
3. 再问：“条件为假时我要得到什么?”写在 : 后面；
4. 如果表达式较长，建议加括号提升可读性。

5.3.5 什么时候适合用

适合：非常简单的二选一赋值或输出值。

```
int absValue = (x >= 0) ? x : -x;
```

不适合：一边要执行很多语句、嵌套很多层、逻辑很复杂。此时使用 if--else 往往更清楚。

5.3.6 “可以继续赋值”的正确理解

下面的代码成立：

```
int a = 1;
int b = 2;
bool chooseA = false;

(chooseA ? a : b) = 9;
```

因为 chooseA 为假，所以整个条件表达式选中 b，赋值后 b=9。

但这不是因为“所有三目运算符都返回变量”，而是因为这里第二、第三个操作数恰好是合适的左值。普通的常量二选一就不能做赋值目标。

一句记忆

条件？真时结果：假时结果。只选一边，只算一边。

6 switch 与三目运算符的区别

比较项目	switch	?:
本质	语句，用于选择控制流	表达式运算符，用于产生一个结果
分支数量	常用于多个离散分支	本质上二选一
典型场景	菜单、编号、字符命令、枚举状态	简单赋值、简单取最大/最小、简单条件结果
是否用 break	常用来阻止贯穿	不使用 break
是否直接表示区间	普通 case 不适合	条件部分可以写范围判断

7 最基础示例汇总

7.1 示例 1：用 switch 选择菜单

```
int menu = 1;
switch (menu) {
    case 1:
        cout << "Start";
        break;
    case 2:
        cout << "Exit";
        break;
    default:
        cout << "Invalid";
        break;
}
```

步骤：先得到 menu=1；与 case 1 相等；输出 Start；遇到 break 后结束 switch。

7.2 示例 2：观察贯穿

```
int n = 2;
switch (n) {
    case 1:
        cout << "1";
        break;
    case 2:
        cout << "2";
    case 3:
```

```
        cout << "3";
        break;
}
```

输出 23。因为匹配 case 2 后没有 break，于是继续执行 case 3 下方的语句。

7.3 示例 3：用三目运算符求较大值

```
int a = 4;
int b = 7;
int larger = (a > b) ? a : b;
```

由于 4 > 7 为假，所以选第三个表达式 b，得到 larger=7。

7.4 示例 4：条件表达式作为赋值目标

```
int x = 10;
int y = 20;
bool flag = true;
(flag ? x : y) = 0;
```

flag=true，因此选择 x 作为赋值目标，最后 x=0、y=20。

8 知识点—题目对应表

题号	题型	对应知识点	难度	主要能力
1	单选	`switch` 的基本用途	入门	概念识别
2	单选	`switch` 基本语法结构	入门	语法辨认
3	单选	常见可用条件类型	入门	类型辨认
4	单选	`case` 的常量值要求	基础	规则辨认
5	单选	`break` 的作用	入门	概念识别
6	单选	`default` 的作用	入门	概念识别
7	单选	fall-through	基础	代码阅读
8	单选	三目运算符语法	入门	语法辨认
9	单选	条件为真时的求值	入门	规则识别
10	单选	三目运算符只求值一个分支	基础	规则理解
11	多选	`switch` 的合法基础形式	基础	综合辨认
12	多选	`break`/`default`	基础	规则判断
13	多选	三目运算符基本性质	基础	规则判断
14	多选	条件表达式作为左值	基础	简单代码理解
15	多选	`switch` 与 `?:` 的用途比较	基础	场景判断
16	填空	防止贯穿的关键字	入门	记忆
17	填空	兜底标签	入门	记忆
18	填空	三目运算符完整结构	入门	记忆
19	填空	条件为假时选取的表达式	入门	规则记忆
20	填空	简单代码输出	基础	代码阅读
21	解答	写基础 `switch`	基础	模仿编程
22	解答	分析无 `break` 的输出	基础	流程分析
23	解答	`if--else` 改写为三目运算符	基础	语法应用
24	解答	`switch`+`?:` 轻度综合	基础	两知识点结合

9 单项选择题（10 道）

说明：每题只有一个正确答案。

1. `switch` 语句最适合完成下列哪一种任务？

A. 根据一个离散值在多个分支中选择一个执行入口

B. 自动把字符串转换成整数

C. 重复执行同一段代码 100 次

D. 定义一个新的变量类型

2. 下列哪一项最接近标准的 `switch` 基本结构？

A. `switch x { if 1: ... }`

B. `switch (x) { case 1: ...; break; }`

C. `switch [x] { case = 1; }`

D. `switch (x): case 1 then ...`

3. 在最基础的 C++ 用法中，下列哪一个值可以直接作为常见的 `switch` 条件？

A. `3.14`

B. `"hello"`

C. 字符 `'A'`

D. 一个 `std::vector<int>`

4. 下列哪个 `case` 标签是最典型、最基础的正确写法？

A. `case x > 10:`

B. `case 2:`

C. `case 1.5:`

D. `case "A":`

5. 在 `switch` 中，`break;` 最常见的作用是什么？

A. 重新执行当前 `case`

B. 退出当前最内层 `switch`

C. 结束整个程序

D. 把条件值改为 0

6. 如果没有任何 `case` 与条件值匹配，而程序中写了 `default:`，通常会怎样？

A. 执行 `default` 后面的语句

B. 自动回到第一个 `case`

C. 编译器必然报错

D. 自动把条件值改成第一个 `case` 的值

7. 下面代码的输出是什么？

```
int x = 1;
switch (x) {
    case 1: cout << "A";
    case 2: cout << "B"; break;
}
```

A. A

B. B

C. AB

D. 无输出

8. 三目运算符的基本语法是哪一个？

A. `condition : expr2 ? expr3`

B. `condition ? expr2 : expr3`

C. `condition ?? expr2 :: expr3`

D. `if ? condition : else`

9. 对表达式 `condition ? A : B`，当 `condition` 为真时，结果来自哪里？

A. `condition`

B. `A`

C. `B`

D. `A` 和 `B` 都会先求值再比较

10. 对最普通的条件表达式 `condition ? expr2 : expr3`，下列说法正确的是哪一项？

A. `expr2` 和 `expr3` 总是都会求值

B. 只有 `expr2` 会求值

C. 只有被条件选中的那一个分支会求值

D. 三目运算符没有结果值

10 多项选择题（5 道）

说明：每题有 2 个或 3 个正确选项。全部选对得分；选错不得分。

11. 关于基础 `switch` 语句，下列说法正确的是哪些？

A. `char` 可作为常见的 `switch` 条件类型

B. `case 1:` 是典型的合法标签

C. 同一个 `switch` 中可以随意写两个相同值的 `case 1:`

D. `case` 一般表示离散的常量值，而不是直接表示运行时区间

12. 关于 `break` 和 `default`，下列说法正确的是哪些？

A. `break` 常用来阻止向后贯穿

B. `default` 不是每个 `switch` 都必须写

C. 没有 `break` 时，后面的标签一定会重新比较条件再决定是否执行

D. 如果最后一个分支已经位于 `switch` 末尾，末尾 `break` 在很多简单情形下可省略

13. 关于表达式 `c ? x : y`，下列说法正确的是哪些？

A. 它有三个操作数

B. `c` 按布尔意义决定选择哪一边

C. `x` 与 `y` 必定都会求值

D. 条件为真时结果来自 `x`

14. 已知：

```
int a = 1;
int b = 2;
bool flag = true;
(flag ? a : b) = 9;
```

下列说法正确的是哪些？

A. 这段赋值在这里是合法的

B. 最后 `a` 的值为 9

C. 最后 `b` 的值为 9

D. 这个例子说明“三目运算符永远都能放在赋值号左边”

15. 关于 `switch` 与三目运算符，下列说法正确的是哪些？

A. `switch` 常用于多个离散分支

B. 三目运算符很适合简单的二选一结果

C. 三目运算符必须配合 `break`

D. 对“分数是否大于等于 90”这种简单区间判断，`if` 或条件表达式通常比普通 `case` 更直接

11 填空题（5 道）

16. 在 `switch` 中，为了在完成一个分支后立即离开当前 `switch`，常写关键字 _____。

17. 当所有 `case` 都不匹配时，可以使用可选标签 _____处理其他情况。

18. 三目运算符的基本结构是：_____。

19. 在三目表达式中，若 `condition` 为假，求值的是：_____。

20. 写出下面代码的输出：_____。

```
int n = 2;
switch (n) {
    case 1: cout << "A"; break;
    case 2: cout << "B"; break;
    default: cout << "C";
}
```

12 基础解答题（4 道）

21. 基本概念应用：写一个最简单的菜单 switch。

有整数变量 choice。要求：当 choice==1 时输出 A；当 choice==2 时输出 B；当 choice==3 时输出 C；其他值输出 Invalid。请只写出核心 switch 结构。

22. 简单步骤题：分析贯穿。

写出下面代码的最终输出，并按执行顺序说明原因。

```
int n = 2;
switch (n) {
    case 1:
        cout << "A";
        break;
    case 2:
        cout << "B";
    case 3:
        cout << "C";
        break;
    default:
        cout << "D";
}
```

23. 三目运算符直接应用。

已有：

```
int a = 6;
int b = 9;
int larger;
```

请用一条三目运算符语句给 larger 赋值，使它等于 a 和 b 中较大的那个，并写出最终 larger 的值。

24. 两个知识点的轻度综合。

阅读下面代码，写出 x 的值与最终输出，并说明两个步骤分别用了哪个知识点。

```
int choice = 2;
int x = 0;

switch (choice) {
    case 1:
        x = 4;
        break;
    case 2:
        x = 8;
        break;
    default:
        x = 1;
}

cout << ((x > 5) ? "High" : "Low");
```

13 答案速查表

单项选择题

题号	1	2	3	4	5	6	7	8	9	10
答案	A	B	C	B	B	A	C	B	B	C

多项选择题

题号	11	12	13	14	15
答案	ABD	ABD	ABD	AB	ABD

填空题

16. `break`

17. `default`

18. `condition ? expr2 : expr3`（等价中文形式“表达式 1 ? 表达式 2 : 表达式 3”也可）

19. `expr3`

20. `B`

解答题结论

21. 见后文标准代码；

22. 输出 `BC`；

23. `larger = (a > b) ? a : b;`，最终 `larger=9`；

24. `x=8`，输出 `High`。

14 单项选择题详细解析

第 1 题

正确答案：A。

考查知识点： `switch` 的基本用途。

定义与规则： `switch` 根据一个条件的离散值，把控制流转移到相应的 `case` 位置。[1][2]

逐项判断：

- A 正确：这正是 `switch` 的典型用途。
- B 错误：`switch` 不负责字符串转整数。
- C 错误：重复执行代码属于循环结构的任务，例如 `for`、`while`。
- D 错误：定义变量类型与 `switch` 无关。

常见误区：把“选择语句”和“循环语句”混淆。

一句总结：`switch` 是“多个分支中选入口”，不是“反复执行”。

第 2 题

正确答案：B。

考查知识点：基础语法。

规则：基本形式是 `switch (condition) { case constant: statements; ... }`。

逐项判断：

- A 错：`switch` 后面需要圆括号；`if 1:` 也不是 C++ 的 `case` 写法。
- B 正确：圆括号、花括号、`case`、冒号与 `break` 的位置均符合基础语法。
- C 错：`switch` 不用方括号，`case = 1;` 也不对。
- D 错：这是接近自然语言/其他语言风格的写法，不是 C++ 语法。

误区：`case` 后面是冒号 `:`，不是等号。

一句总结：`switch（值）{ case 常量: ... }`。

第 3 题

正确答案：C。

考查知识点：`switch` 的条件类型。

规则：标准规定条件可为整数类型、枚举类型，或按规则可转换到这些类型的类类型。[1] `char` 属于整数类型，因此 `'A'` 是典型可用条件值。

逐项判断：

- A 错：`3.14` 是浮点数，普通 `switch` 不以浮点值直接匹配 `case`。
- B 错：字符串字面量不是普通 `switch` 的直接离散条件类型。
- C 正确：字符是最常见的入门示例之一。
- D 错：`std::vector<int>` 不能按这种基本方式作为 `switch` 条件。

误区：“字符”看起来不像整数，但在 C++ 类型体系中 `char` 是整数类型家族成员。

一句总结：入门先记“整数、字符、枚举等离散值”。

第 4 题

正确答案：B。

考查知识点：`case` 标签。

规则：`case` 后面使用与 `switch` 条件类型相容的转换常量表达式。[1]

逐项判断：

- A 错：`x > 10` 是一个运行时条件写法，不是这里用于表示区间的普通 `case` 标签。
- B 正确：`2` 是最基础的整数常量。
- C 错：`1.5` 是浮点常量，不属于这里的基础合法形式。
- D 错：`"A"` 是字符串字面量，不是字符 `'A'`；普通 `case` 不这样写。

误区：单引号 `'A'` 与双引号 `"A"` 的类型不同。

一句总结：`case` 列的是一个个“固定点值”。

第 5 题

正确答案：B。

考查知识点：`break`。

规则：在 `switch` 中，`break` 终止最内层的 `switch`，控制流转到其后。[1][4]

逐项判断：

- A 错：`break` 不会重新执行当前分支。
- B 正确。
- C 错：它不等于结束整个程序；结束整个程序通常不是 `break` 的语义。
- D 错：`break` 不修改条件值。

误区：把“退出一个控制结构”和“结束整个程序”混淆。

一句总结：在这里，`break` 的关键词就是“跳出当前 switch”。

第 6 题

正确答案：A。

考查知识点：`default`。

规则：没有 `case` 匹配时，如果有 `default`，控制流进入 `default`。若也没有 `default`，则不会执行任何对应分支。[1]

逐项判断：

- A 正确。
- B 错：不会自动回到第一项。
- C 错：没有匹配值不是编译错误。
- D 错：条件值不会被自动修改。

误区：误认为 `default` 相当于“自动给条件赋一个默认值”。它只是一个兜底入口。

一句总结：`default` 管“其他所有未匹配情况”。

第 7 题

正确答案：C，即输出 `AB`。

考查知识点：贯穿（fall-through）。

逐步判断：

1. `x=1`，匹配 `case 1`；
2. 输出 `A`；
3. `case 1` 后没有 `break`；
4. 控制流继续向下，进入 `case 2` 后面的语句；
5. 输出 `B`；
6. 遇到 `break`，退出 `switch`。

选项判断：A 少算了贯穿；B 忽略了先执行 `case 1`；C 正确；D 显然与执行路径不符。

误区：以为到 `case 2` 标签时会再次判断 `x==2`。不会；已经进入 `switch` 后，标签本身不会阻止顺序执行。

一句总结：匹配一次后，没 `break` 就可能继续往下走。

第 8 题

正确答案：B。

考查知识点：三目运算符语法。

规则：`condition ? expr2 : expr3`。[1][3]

逐项判断：只有 B 的问号与冒号顺序正确，其余三项都不是 C++ 条件运算符语法。

误区：把 ？ 与 ： 的顺序记反。

一句总结：“条件问一问，真值在问号后，假值在冒号后”。

第 9 题

正确答案：B。

考查知识点：三目运算符求值规则。

规则：条件为真，整个条件表达式的结果来自第二个操作数；为假则来自第三个操作数，而且只求值被选中的那一个。[1][3]

逐项判断：A 是条件本身，不是返回分支；B 正确；C 是假分支；D 错在“都先求值”。

一句总结：真选第二项，假选第三项。

第 10 题

正确答案：C。

考查知识点：短路式分支求值。

规则：标准明确说明第二、第三个操作数中只求值一个。[1][3]

逐项判断：

- A 错：不会两边都算。
- B 错：当条件为假时求值第三项。
- C 正确。
- D 错：条件运算符是表达式，本身有结果（具体类型由规则决定）。

一句总结：?：是真正的二选一，不是两边都执行后再挑结果。

15 多项选择题详细解析

第 11 题

正确答案：A、B、D。

A：正确。 `char` 是整数类型，可作为典型的 `switch` 条件。[1]

B：正确。 `case 1:` 是最基础、最常见的常量标签。

C：错误。同一个 `switch` 中，转换后具有相同值的两个 `case` 常量不允许重复。[1]

D：正确。普通 `case` 是离散常量点，不是运行时区间条件。

误区：认为 `case` 只是“一个 if 的替代品”，所以可以任意写布尔条件。基础 `switch` 不是这样工作的。

第 12 题

正确答案：A、B、D。

A：正确。 `break` 常用来阻止贯穿。

B：正确。 `default` 是可选的。[1]

C：错误。没有 `break` 时不是“再判断标签条件”，而是控制流继续向下。

D：正确。如果已经到整个 `switch` 末尾，程序自然会离开；末尾 `break` 常可省略。初学阶段保留也没有问题。

误区：把 `case` 标签理解成一道“自动门”。实际上标签只是可能的跳转入口。

第 13 题

正确答案：A、B、D。

A：正确。条件运算符有三个操作数，所以称三目运算符。

B：正确。第一个操作数按规则转换/解释为布尔条件。[1][3]

C：错误。第二、第三项只会求值一个。

D：正确。条件为真时结果来自第二项 `x`。

误区：“表达式写在代码里”不等于“每个表达式都会执行”。

第 14 题

正确答案：A、B。

A：正确。这里 `a` 与 `b` 都是同类型可修改左值；条件表达式在该情形下可作为赋值目标。[1][3]

B：正确。 `flag=true`，选择 `a`，因此执行效果等价于 `a=9`。

C：错误。 `b` 没被选中，仍然是 2。

D：错误。不能推广成“所有三目表达式都能放赋值号左边”。例如 `(flag ? 1 : 2) = 9;` 不成立。

误区：把一个成立的特例误记成无条件规则。

第 15 题

正确答案：A、B、D。

A：正确。 `switch` 典型用途就是多个离散分支。

B：正确。三目运算符适合简单二选一结果。

C：错误。 `break` 是控制语句，不是三目运算符语法组成部分。

D：正确。“大于等于 90”是范围/关系条件，直接用 `if` 或 `?:` 的条件部分更自然。

误区：把“都能做选择”误解成“语法和使用场景完全相同”。

16 填空题详细解析

第 16 题

标准答案： `break`。

规则： 在 `switch` 中，`break;` 终止当前最内层 `switch`。[1][4]

推理： 题目要求“完成分支后立即离开”，对应的基础关键字就是 `break`。

格式要求： 填 `break` 即可；若写完整语句 `break;` 也可接受。

易错点： 不要写 `continue`；`continue` 的典型用途在循环中。

第 17 题

标准答案： `default`。

规则： `default` 是没有 `case` 匹配时的可选兜底标签。[1]

易错点： 不要误写成 `else`。普通 `switch` 使用的是 `default:`。

第 18 题

标准答案： `condition ? expr2 : expr3`。

可接受等价形式：**表达式 1 ? 表达式 2 : 表达式 3**。

规则： 问号把条件与真分支分开，冒号把真分支与假分支分开。

易错点： 最常见错误是把问号和冒号位置写反。

第 19 题

标准答案： `expr3`。

规则： 条件为假时只求值第三个操作数，并以它的结果作为条件表达式结果。[1][3]

推理： 公式 `condition ? expr2 : expr3` 可以直接记为“真二假三”。

第 20 题

标准答案： `B`。

步骤：

1. `n=2;`
2. 匹配 `case 2:`;
3. 执行 `cout << "B";`;
4. 立刻遇到 `break;`，退出 `switch`;

5. 因此既不会执行 `case 1`，也不会执行 `default`。

易错点：不要把 `default` 理解为“最后一定会执行”。它只在没有任何 `case` 匹配时才作为入口。

17 基础解答题详细解析

第 21 题：写基础 switch

题意分析：已知一个整数 choice，要求对值 1、2、3 分别输出 A、B、C，其他值输出 Invalid。

知识定位：离散的 1、2、3 三个值对应典型的 switch--case 场景。

规则说明：

- switch(choice)：取得要判断的离散值；
- case 1:、case 2:、case 3:：分别给出三个常量入口；
- break;：完成当前分支后离开；
- default:：处理 1、2、3 之外的值。

标准答案：

```
switch (choice) {
    case 1:
        cout << "A";
        break;
    case 2:
        cout << "B";
        break;
    case 3:
        cout << "C";
        break;
    default:
        cout << "Invalid";
        break;
}
```

结果检查：分别假设 choice=1,2,3,4，应该得到 A、B、C、Invalid。每个 case 后都有 break，不会意外贯穿。

易错提醒：case 后写冒号；不要把 default 写成 else。

方法总结：离散值有几个，就列几个 case；其他情况统一交给 default。

第 22 题：分析贯穿

题意分析：给定 n=2，要求跟踪 switch 的控制流。

知识定位：case 匹配 + 缺少 break 后的 fall-through。

解题步骤：

1. 条件值为 2，因此直接跳到 case 2:；
2. 执行 cout << "B";，先输出 B；

3. case 2 后没有 break;；

4. 控制流继续向下，不重新判断 n==3，直接执行 case 3 下方的 cout << "C";；

5. 输出 C；

6. 遇到 break;，退出 switch；

7. default 不执行。

规范答案：最终输出为 BC。

结果检查：如果在 case 2 输出 B 后补上 break;，则输出会变成只有 B。这说明原结果中的 C 正是由贯穿造成的。

易错提醒：不要认为执行到 case 3 时还会再次检查 n 是否为 3。

方法总结：分析 switch 时先找“从哪个标签进入”，再从那里按顺序往下追踪，直到遇到离开语句或结构末尾。

第 23 题：三目运算符直接应用

题意分析：在 a=6、b=9 中取较大值赋给 larger。

知识定位：二选一结果最适合用 ?:。

公式/规则：

$$\text{condition?expr2 : expr3}$$

其中：

- condition：判断条件，本题写 a > b；
- expr2：条件为真时的结果，本题取 a；
- expr3：条件为假时的结果，本题取 b。

标准答案：

```
1 larger = (a > b) ? a : b;
```

代入判断：6 > 9 为假，因此选择 b，得到 larger=9。

结果检查：9 确实是 6 和 9 中较大的数。

易错提醒：如果写成 (a > b) ? b : a，真假分支顺序被写反，就会选到较小值。

方法总结：先写比较条件，再把“真时答案”和“假时答案”分别放在问号后与冒号后。

第 24 题：switch + 三目运算符

题意分析：代码分两阶段：先用 switch 决定 x，再用三目运算符根据 x>5 输出 High 或 Low。

知识定位：

1. 第一阶段：离散值 1、2、其他，使用 switch；

2. 第二阶段：只有 High/Low 两种结果，使用 `?:`。

步骤 1：执行 switch。

`choice=2`，匹配 `case 2:`，执行 `x=8;`，随后 `break;` 离开。因此：

$$\boxed{x = 8}$$

步骤 2：执行三目运算符。

判断 `x > 5`，也就是 `8 > 5`，结果为真，所以选第二个表达式 `"High"`。

最终答案：

$$\boxed{x = 8, \quad \text{输出为} \texttt{High}}$$

结果检查：如果 `choice=1`，则 `x=4`，第二阶段会输出 Low；这与条件 `x>5` 的含义一致。

易错提醒：两个选择结构不要混在一起追踪。先把 `switch` 完全执行完，确定 `x`，再判断三目表达式。

方法总结：复杂度很低的组合题可以拆成“先离散多分支，再二选一结果”两个独立步骤。

18 高频易错点总结

1. **把 switch 条件范围记得过窄**。不要死记“只能 int/char”；更准确的是整数、枚举及特定可转换类类型。入门阶段重点掌握整数/字符/枚举等离散值。
2. **把 case 写成范围**。`case score >= 90`：不是普通区间分支的正确思路；区间更适合 `if`。
3. **忘记 break**。会导致继续执行后面的语句，而不是“只执行当前 case”。
4. **误解 fall-through**。后续 case 不会重新判断是否匹配；标签不会自动阻断顺序执行。
5. **误认为 default 必须有**。它是可选兜底分支。
6. **误认为三目两边都会执行**。实际只求值被条件选中的一个分支。[1][3]
7. **把“可作为左值”的特例泛化**。`(flag ? a : b)=9;` 这种写法有条件，不代表所有三目表达式都能被赋值。
8. **为了短而滥用三目运算符**。简单二选一适合，复杂多语句逻辑更适合清晰的 `if--else`。

19 基础解题步骤总结

19.1 看到 switch 题时

1. 先计算/确定 switch 条件的值；
2. 找到值相同的 case；
3. 从匹配的标签后开始顺序执行；
4. 每执行一段都观察是否遇到 break 或其他离开控制流的语句；
5. 若无匹配，检查是否有 default。

19.2 看到三目运算符时

1. 先只算第一个条件；
2. 条件真：只看问号后、冒号前的表达式；
3. 条件假：只看冒号后的表达式；
4. 不要同时计算真、假两边；
5. 最后把被选中的结果代回赋值或输出位置。

20 参考资料与核验来源

本资料的知识核验优先使用 C++ 标准委员会工作草案与编译器厂商官方文档。关键结论至少以标准草案与 Microsoft Learn 进行交叉核对。

[1] ISO/IEC JTC 1/SC 22/WG21. *N5054: Working Draft, Programming Languages — C++*. 2026-07-16. 重点核验：`switch` 条件类型、`case` 常量要求、`default`、fall-through、`break`、条件运算符 `?:` 的求值规则和值类别。
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/n5054.pdf
访问日期：2026-08-19。

[2] Microsoft Learn. *switch statement (C++)*. 重点核验：`switch` 基础语义、`case` 匹配、没有 `break` 时的 fall-through 示例。
https://learn.microsoft.com/en-us/cpp/cpp/switch-statement-cpp?view=msvc-170
访问日期：2026-08-19。

[3] Microsoft Learn. *Conditional Operator: ?:*. 2024-03-01 页面版本。重点核验：三目运算符三个操作数、第一操作数按布尔意义判断、第二和第三操作数只求值其一、特定情形下的 lvalue 结果。
https://learn.microsoft.com/en-us/cpp/cpp/conditional-operator-q?view=msvc-170
访问日期：2026-08-19。

[4] Microsoft Learn. *break Statement (C++)*. 2024-01-25 页面版本。重点核验：`break` 在 `switch` 中终止当前结构并转移到其后语句。
https://learn.microsoft.com/en-us/cpp/cpp/break-statement-cpp?view=msvc-170
访问日期：2026-08-19。

21 最终质量检查记录

检查项目	结果
图片主要内容是否重新编码	已完成；两张图片中的标题、语法、注意事项、红字补充和三目运算符特点均已纳入
明显错误/不严谨处是否说明	已完成；共重点说明 6 类问题
单项选择题数量	10 道，已核对
多项选择题数量	5 道，已核对
填空题数量	5 道，已核对
基础解答题数量	4 道，已核对
总题量	24 道，已核对
难度	全部为入门/基础；无竞赛、压轴、复杂参数题
答案位置	全部题目之后统一给出
单选错误选项解析	每题均说明
多选逐项解析	A/B/C/D 均逐项说明
公式/语法解释	首次使用处均说明组成、用途与适用情形
参考来源	采用 WG21 工作草案与 Microsoft Learn 官方文档交叉核验
LaTeX 编译	使用 XeLaTeX 实际编译并在交付前检查

最终记忆卡片

- `switch`：适合“一个离散值，对应多个分支”。
- `case`：列固定常量值；普通写法不直接表达区间。
- `break`：常用来阻止向下贯穿。
- `default`：可选，负责其他未匹配值。
- `?:`：`条件? 真时结果: 假时结果`。
- `?:`：第二、第三项只求值被选中的一项。
- “三目结果能继续赋值”只在特定左值条件下成立，不是无条件规则。